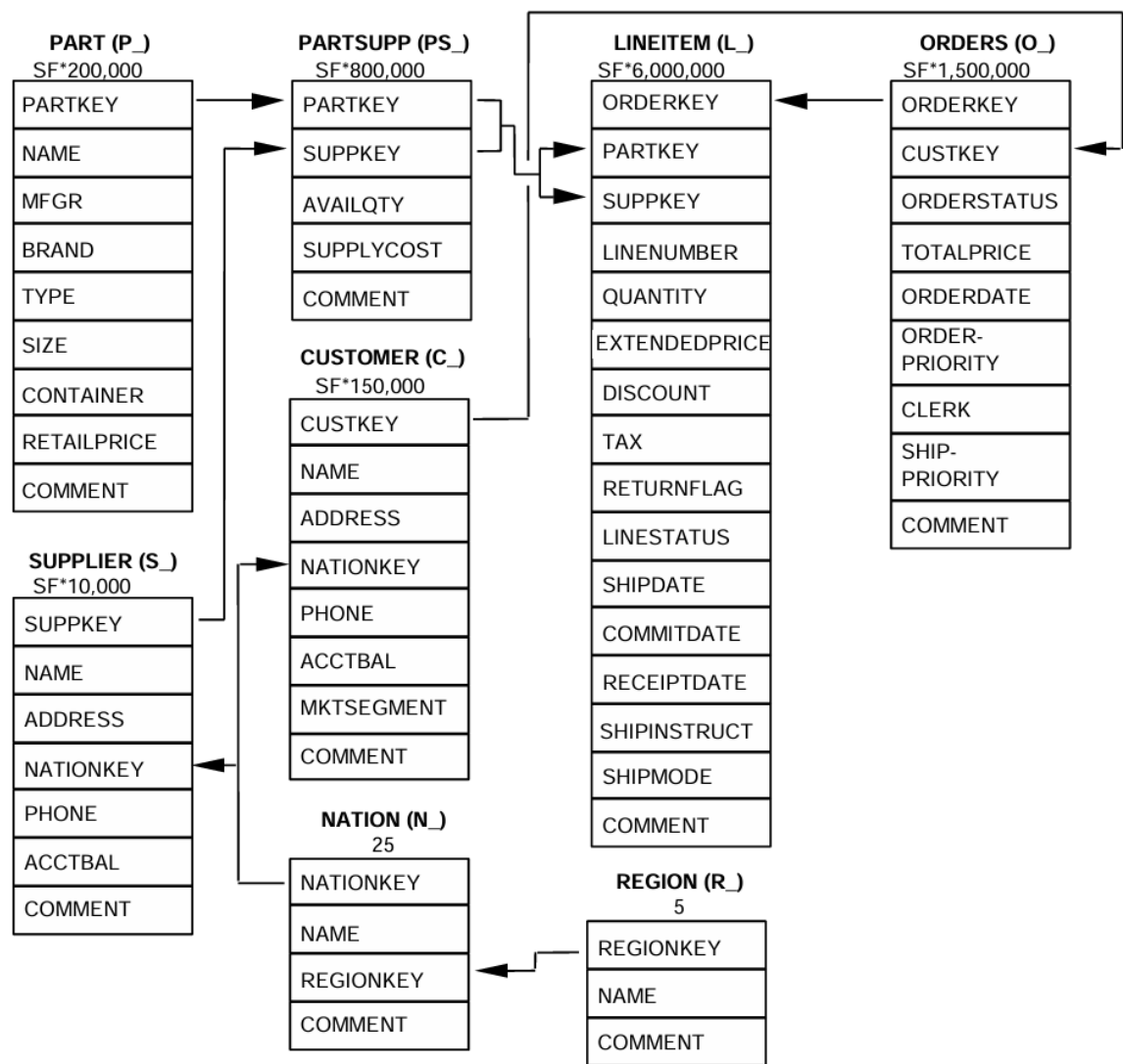


TPCH数据使用说明

表模式：

Figure 2: The TPC-H Schema



TPCH官网链接: [TPC-H Homepage](https://www.tpc-h.org/)

同学们可以自行下载TPCH-tools自行生成数据（10GB），下面主要是以mysql为例说明TPC-H的使用方法。

供同学自行参考：

[windows：TPC-H测试的数据库模式生成及其数据导入MySQL的教程记录](#)

[postgresql使用（一）：TPC-H tools生成数据集并导入至postgre的数据库 tpch-tools](#)

生成好的10GB的TPC-H数据: <https://www.alipan.com/s/Ttu3gGjrHDc>

其实只要生成好数据文件，通过数据库自带的load或者copy命令导入就可以了；如果想手动导入可以首先通过dss.ddl 和 dss.ri里面的语句进行表的创建和约束的创建，然后使用load/copy导入数据即可

以下教程适用于Linux系统下MySQL数据库导入TPC-H数据。

如果是windows下的kingbase系统，请参考5.Kingbase导入TPC-H数据

1. 下载和修改

将下载得到的安装包上传到服务器上，重命名后解压：

```
unzip tpc-h.zip
```

后续大部分的 修改、编译、执行等操作都在 `tpc-h/dbgen` 目录下完成

```
cd mysql-tpc-h/dbgen
```

1.1 修改makefile文件

该文件约束了 编译 后可执行文件的部分条件以及需要的依赖。

```
cp makefile.suite makefile
vim makefile

# 修改 103 109 110 111 这四行的内容
CC = gcc
# Current values for DATABASE are: INFORMIX, DB2, TDAT (Teradata)
#                                SQLSERVER, SYBASE, ORACLE, VECTORWISE
# Current values for MACHINE are: ATT, DOS, HP, IBM, ICL, MVS,
#                                SGI, SUN, U2200, VMS, LINUX, WIN32
# Current values for WORKLOAD are: TPCB
# 如果你是MySQL数据库
DATABASE= MYSQL
MACHINE = LINUX
WORKLOAD = TPCB

# 如果你是PG数据库
DATABASE= POSTGRESQL
MACHINE = LINUX
WORKLOAD = TPCB
```

后续都以mysql为例进行说明，如果你是postgres，可以参考[postgresql使用（一）：TPC-H tools生成数据集并导入至postgre的数据库 tpch-tools](#)

或者其他数据库，可以自行寻找相关资料。

1.2 修改tpc-d.h文件

```
// MySQL
#ifdef MYSQL
#define GEN_QUERY_PLAN ""
#define START_TRAN "START TRANSACTION"
#define END_TRAN "COMMIT"
#define SET_OUTPUT ""
#define SET_ROWCOUNT "limit %d;\n"
#define SET_DBASE "use %s;\n"
#endif

// PG
```

```
#ifdef POSTGRESQL
#define GEN_QUERY_PLAN "EXPLAIN"
#define START_TRAN "BEGIN TRANSACTION"
#define END_TRAN "COMMIT"
#define SET_OUTPUT ""
#define SET_ROWCOUNT "LIMIT %d\n"
#define SET_DBASE ""
#endif
```

1.3 编译

在dbgen目录下进行编译，可能需要安装gcc-c++

```
make
# 或者
make -f makefile.suite
```

2. TPC-H 数据的生成、导入及查询生成

2.1 生成 `<table>.tbl` 数据文件

可以使用 `./dbgen -h` 命令查看，也可以查看 `README` 中对 `dbgen` 的相关介绍。

```

$ ./dbgen -h
TPC-H Population Generator (Version 3.0.0 build 0)
Copyright Transaction Processing Performance Council 1994 - 2010
USAGE:
dbgen [-{vf}][{-T {pcsoPSOL}}]
      [-s <scale>][{-C <procs>}][-S <step>]
dbgen [-v] [-O m] [-s <scale>] [-U <updates>]

Basic Options
=====
-C <n> -- separate data set into <n> chunks (requires -S, default: 1)
-f      -- force. Overwrite existing files
-h      -- display this message
-q      -- enable QUIET mode
-s <n> -- set Scale Factor (SF) to <n> (default: 1)
-S <n> -- build the <n>th step of the data/update set (used with -C or -U)
-U <n> -- generate <n> update sets
-v      -- enable VERBOSE mode

Advanced Options
=====
-b <s> -- load distributions for <s> (default: dists.dss)
-d <n> -- split deletes between <n> files (requires -U)
-i <n> -- split inserts between <n> files (requires -U)
-T c   -- generate customers ONLY
-T l   -- generate nation/region ONLY
-T L   -- generate lineitem ONLY
-T n   -- generate nation ONLY
-T o   -- generate orders/lineitem ONLY
-T O   -- generate orders ONLY
-T p   -- generate parts/partsupp ONLY
-T P   -- generate parts ONLY
-T r   -- generate region ONLY
-T s   -- generate suppliers ONLY
-T S   -- generate partsupp ONLY

To generate the SF=1 (1GB), validation database population, use:
    dbgen -vf -s 1

To generate updates for a SF=1 (1GB), use:
    dbgen -v -U 1 -s 1

```

使用命令生成符合基准的8张表与相应的数据。由介绍可知，`-s` 后面是数据量的大小，若为 **10** 则生成 **10GB** 数据，若为**100**则生成**100GB**的数据。

-V：显示进度消息

-f：覆盖原始数据

-s：生成数据的总体规模

```

# 生成数据
./dbgen -vf -s 10
# 查看生成的数据 应该是8张表的数据
ls *.tbl

```

2.2 导入构建脚本文件

编译之后有 `dss.ddl` 和 `dss.ri` 两个脚本，`dss.ddl`是建表脚本，`dss.ri`脚本是建立主键外键关联关系。

因为TPC-H benchmark本身不支持MySQL；并且由于MySQL的表名、库名等默认都是严格区分大小写的，而 `dss.ddl`（建表脚本）与 `dss.ri`（表约束脚本）中使用的都是大写的数据库名与表名，但是提供的 22 条查询语句又使用的小写的表名。所以需要先修改这两个脚本。

修改 dss.ddl

在文件开头添加以下语句：

```
DROP DATABASE IF EXISTS tpch;
CREATE DATABASE tpch;
USE tpch;
```

修改 dss.ri

```
-- Scsid: @(#)dss.ri 2.1.8.1
-- tpch Benchmark version 8.0

-- CONNECT TO tpch;
USE tpch;

-- ALTER TABLES TO DROP PRIMARY KEYS
-- ALTER TABLE tpch.REGION DROP PRIMARY KEY;
-- ALTER TABLE tpch.NATION DROP PRIMARY KEY;
-- ALTER TABLE tpch.PART DROP PRIMARY KEY;
-- ALTER TABLE tpch.SUPPLIER DROP PRIMARY KEY;
-- ALTER TABLE tpch.PARTSUPP DROP PRIMARY KEY;
-- ALTER TABLE tpch.ORDERS DROP PRIMARY KEY;
-- ALTER TABLE tpch.LINEITEM DROP PRIMARY KEY;
-- ALTER TABLE tpch.CUSTOMER DROP PRIMARY KEY;

-- For table REGION
ALTER TABLE tpch.REGION ADD PRIMARY KEY (R_REGIONKEY);

-- For table NATION
ALTER TABLE tpch.NATION ADD PRIMARY KEY (N_NATIONKEY);
ALTER TABLE tpch.NATION
    -- ADD FOREIGN KEY NATION_FK1 (N_REGIONKEY) references tpch.REGION;
    ADD FOREIGN KEY NATION_FK1 (N_REGIONKEY) REFERENCES tpch.REGION
(R_REGIONKEY);
COMMIT WORK;

-- For table PART
ALTER TABLE tpch.PART ADD PRIMARY KEY (P_partKEY);
COMMIT WORK;

-- For table SUPPLIER
ALTER TABLE tpch.SUPPLIER ADD PRIMARY KEY (S_SUPPKEY);
ALTER TABLE tpch.SUPPLIER ADD FOREIGN KEY supplier_FK1 (S_NATIONKEY) REFERENCES
tpch.NATION (N_NATIONKEY);
COMMIT WORK;
```

```

-- For table PARTSUPP
ALTER TABLE tpch.PARTSUPP ADD PRIMARY KEY (PS_partKEY, PS_SUPPKEY);
COMMIT WORK;

-- For table CUSTOMER
ALTER TABLE tpch.CUSTOMER ADD PRIMARY KEY (C_CUSTKEY);
ALTER TABLE tpch.CUSTOMER ADD FOREIGN KEY customer_FK1 (C_NATIONKEY) REFERENCES
tpch.NATION (N_NATIONKEY);
COMMIT WORK;

-- For table LINEITEM
ALTER TABLE tpch.LINEITEM ADD PRIMARY KEY (L_ORDERKEY, L_LINENUMBER);
COMMIT WORK;

-- For table ORDERS
ALTER TABLE tpch.ORDERS ADD PRIMARY KEY (O_ORDERKEY);
COMMIT WORK;

-- For table PARTSUPP FOREIGN KEYS
ALTER TABLE tpch.PARTSUPP ADD FOREIGN KEY partSUPP_FK1 (PS_SUPPKEY) REFERENCES
tpch.SUPPLIER (S_SUPPKEY);
COMMIT WORK;
ALTER TABLE tpch.PARTSUPP ADD FOREIGN KEY partSUPP_FK2 (PS_partKEY) REFERENCES
tpch.PART (P_partKEY);
COMMIT WORK;

-- For table ORDERS FOREIGN KEYS
ALTER TABLE tpch.ORDERS ADD FOREIGN KEY orders_FK1 (O_CUSTKEY) REFERENCES
tpch.CUSTOMER (C_CUSTKEY);
COMMIT WORK;

-- For table LINEITEM FOREIGN KEYS
ALTER TABLE tpch.LINEITEM ADD FOREIGN KEY lineitem_FK1 (L_ORDERKEY) REFERENCES
tpch.ORDERS (O_ORDERKEY);
COMMIT WORK;
ALTER TABLE tpch.LINEITEM ADD FOREIGN KEY lineitem_FK2 (L_partKEY, L_SUPPKEY)
REFERENCES tpch.PARTSUPP (PS_partKEY, PS_SUPPKEY);

-- Rename tables to lowercase
ALTER TABLE CUSTOMER RENAME TO customer;
ALTER TABLE LINEITEM RENAME TO lineitem;
ALTER TABLE NATION RENAME TO nation;
ALTER TABLE ORDERS RENAME TO orders;
ALTER TABLE PART RENAME TO part;
ALTER TABLE PARTSUPP RENAME TO partsupp;
ALTER TABLE REGION RENAME TO region;
ALTER TABLE SUPPLIER RENAME TO supplier;

COMMIT WORK;

```

修改 MySQL 配置文件

在建表之前我们需要先修改MySQL的一些配置。一些版本的mysql对通过文件导入导出作了限制，默认不允许。

查看和load data相关的参数配置，可以执行以下MySQL命令：

```
SHOW VARIABLES LIKE "secure_file_priv";
```

如果value值为null，则为禁止；如果有文件夹目录，则只允许改目录下文件（测试子目录也不行）；如果为空，则不限制目录。

修改MySQL配置文件 `my.cnf`，查看是否有 `secure_file_priv=` 这一行的内容，如果没有则需要手动添加。

如何查看my.cnf的配置文件路径呢？

```
mysql --help | grep my.cnf  
#输出: /etc/my.cnf /etc/mysql/my.cnf ~/.my.cnf
```

Windows下的同学可以自行寻找mysql对应的配置文件并修改

修改my.cnf，即添加 `secure_file_priv=` 到文件最后一行，表示不限制目录。**等号一定要有，否则mysql无法启动。**修改完配置文件后，重启MySQL生效。

```
#  
# * IMPORTANT: Additional settings that can override those from this file!  
# The files must end with '.cnf', otherwise they'll be ignored.  
#  
# 原本的  
!includedir /etc/mysql/conf.d/  
!includedir /etc/mysql/mysql.conf.d/  
# 下面是新添加的，要有[mysqld]来标识  
[mysqld]  
secure_file_priv=
```

修改完成之后需要重启mysql，即：`sudo service mysql restart`

这一步是为了方便后面load data，或者你可以把对应的tbl文件移动到/var/lib/下，这样就可以不用修改配置文件，直接load。

但是强烈推荐通过修改 my.cnf 配置文件来完成功能，因为后续参数调优会经常修改该配置文件。

修改完配置文件需要重启mysql才能生效

2.3 生成数据表

开启MySQL服务，登录MySQL，使用命令 `\. path-to-tpc-h/dbgen/dss.dd1` 或者 `source path-to-tpc-h/dbgen/dss.dd1` 来使用修改后的 `dss.dd1` 文件创建数据表。（自行替换你的目录）

执行成功后查看数据库会发现有一个名叫 `tpch` 的数据库；切换到数据库下可以看到由8张表。

```
mysql> use tpch;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
+-----+
| Tables_in_tpch |
+-----+
| customer        |
| lineitem         |
| nation          |
| orders          |
| part            |
| partsupp        |
| region          |
| supplier        |
+-----+
8 rows in set (0.00 sec)
```

建立外键约束

因为该实验主要是进行查询，不需要数据更新，这里也可以不建立这些约束，因为这些约束会严重影响 load data 的速度。你可以选择最后运行该文件，或者直接不运行。如果不运行且你是 MySQL 的话可能需要把表名改成小写的形式(参考 dss.ri 文件里面 Rename tables to lowercase 的部分)，或者你直接修改查询里面的表名都为大写 (MySQL 会区分大小写)

与上述同理，使用命令 `\. path-to-tpc-h/dbgen/dss.ri` 或者 `source path-to-tpc-h/dbgen/dss.ri` 来使用修改后的 `dss.ri` 文件创建表约束。

如果想查看关联关系是否建立成功，可以通过如命令 `show create table customer\G`；查看约束是否创建成功。

2.4 导入tbl数据文件

共需导入8个表，part、region、nation、customer、supplier、orders、partsupp、lineitem，大表的导入时间很长。

一共8张表，先来导入前5张表（以下语句在mysql中执行），前5张都是小表，可以直接导入。

```
LOAD DATA INFILE "/home/TPC-H/dbgen/part.tbl"
INTO TABLE part
FIELDS TERMINATED BY "|";

LOAD DATA INFILE "/home/TPC-H/dbgen/region.tbl"
INTO TABLE region
FIELDS TERMINATED BY "|"
LINES TERMINATED BY "|\n";

LOAD DATA INFILE "/home/TPC-H/dbgen/nation.tbl"
INTO TABLE nation
FIELDS TERMINATED BY "|";

LOAD DATA INFILE "/home/TPC-H/dbgen/customer.tbl"
```

```

INTO TABLE customer
FIELDS TERMINATED BY "|";

LOAD DATA INFILE "/home/TPC-H/dbgen/supplier.tbl"
INTO TABLE supplier
FIELDS TERMINATED BY "|"
LINES TERMINATED BY "|\n";

# 后面三个表最好不要直接load
LOAD DATA INFILE "/home/TPC-H/dbgen/orders.tbl.11"
INTO TABLE orders
FIELDS TERMINATED BY "|";

LOAD DATA INFILE "/home/TPC-H/dbgen/partsupp.tbl"
INTO TABLE partsupp
FIELDS TERMINATED BY "|";

LOAD DATA INFILE "/home/TPC-H/dbgen/lineitem.tbl"
INTO TABLE lineitem
FIELDS TERMINATED BY "|";

# kingbase和pg通过copy来加载
copy region from '/var/lib/postgresql/data/region.tbl' with delimiter as '|' NULL
'';
copy nation from '/var/lib/postgresql/data/nation.tbl' with delimiter as '|' NULL
'';
copy partsupp from '/var/lib/postgresql/data/partsupp.tbl' with delimiter as '|'
NULL '';
copy customer from '/var/lib/postgresql/data/customer.tbl' with delimiter as '|'
NULL '';
copy lineitem from '/var/lib/postgresql/data/lineitem.tbl' with delimiter as '|'
NULL '';
copy orders from '/var/lib/postgresql/data/orders.tbl' with delimiter as '|' NULL
'';
copy part from '/var/lib/postgresql/data/part.tbl' with delimiter as '|' NULL '';
copy supplier from '/var/lib/postgresql/data/supplier.tbl' with delimiter as '|'
NULL '';

```

再来看另外三张表，注意：这里最好不要直接执行。因为这三张表比较大，最好是使用提供的脚本拆解执行。如果直接导入可能需要数个小时，但是使用脚本可以大大提升速度，这里主要的瓶颈是外键检查和唯一性校验的代价较高，可以暂时关闭，load完数据之后再打开。

这里不用脚本导入orders表(行)会使用约四个半小时，而使用脚本导入lineitem只需要约30分钟（20GB数据下测试）。

在dbgen目录下，提供了一个脚本，load_data.sh,主要是通过拆解表以及暂时关闭外键检查来提高导入的速度

只需要把该脚本放在dbgen目录下，依次执行即可。

```
# 确保脚本具有执行权限
chmod +x load_data.sh

./load_data.sh orders
./load_data.sh partsupp
./load_data.sh lineitem
```

查看数据导入情况

```
SELECT 'region' AS TABLE_NAME, COUNT(*) AS TABLE_ROWS FROM region
UNION ALL
SELECT 'nation', COUNT(*) FROM nation
UNION ALL
SELECT 'supplier', COUNT(*) FROM supplier
UNION ALL
SELECT 'customer', COUNT(*) FROM customer
UNION ALL
SELECT 'part', COUNT(*) FROM part
UNION ALL
SELECT 'partsupp', COUNT(*) FROM partsupp
UNION ALL
SELECT 'orders', COUNT(*) FROM orders
UNION ALL
SELECT 'lineitem', COUNT(*) FROM lineitem;
```

输出大致如下:

```
mysql> SELECT 'region' AS TABLE_NAME, COUNT(*) AS TABLE_ROWS FROM region
-> UNION ALL
-> SELECT 'nation', COUNT(*) FROM nation
-> UNION ALL
-> SELECT 'supplier', COUNT(*) FROM supplier
-> UNION ALL
-> SELECT 'customer', COUNT(*) FROM customer
-> UNION ALL
-> SELECT 'part', COUNT(*) FROM part
-> UNION ALL
-> SELECT 'partsupp', COUNT(*) FROM partsupp
-> UNION ALL
-> SELECT 'orders', COUNT(*) FROM orders
-> UNION ALL
-> SELECT 'lineitem', COUNT(*) FROM lineitem;

+-----+-----+
| TABLE_NAME | TABLE_ROWS |
+-----+-----+
| region      |          5 |
| nation      |         25 |
| supplier    |       100000 |
| customer    |     1500000 |
| part        |     2000000 |
| partsupp    |     8000000 |
| orders      |    15000000 |
| lineitem    |    59994608 |
+-----+-----+
```

```
8 rows in set (1 min 26.79 sec)
```

3. 查询生成

前面介绍了 DBGEN，是用来生成表与数据的，现在来介绍 QGEN，这也是编译后生成的可执行文件，主要是用来生成查询 SQL 语句的，同样的，和dbgen一样，可以再README中查看该命令的介绍。

按照下面的脚本，在dbgen目录下执行即可生成查询

```
cp dists.dss qgen queries/
cd queries/

for i in {1..22}
do
    sed -i "/^:n./d" $i.sql
    ./qgen -d $i > d$i.sql
done

for i in {1..22}
do
    sed -i "/^:n./d" $i.sql
    ./qgen -d $i > c$i.sql
done
```

注意：在生成查询语句前，queries文件夹中的 22 条未格式化的SQL语句 模板文件不要改动，否则 qgen 会读取错误导致无法生成正确的查询语句文件

运行脚本后，可以看到现在已经有了 22 个SQL脚本，分别存放的是 22 条测试语句，但是这些测试语句还不能直接使用，需要做些许修改。

```
# 将所有 limit -1 替换为 limit 1
sed -i '$s/limit -1/limit 1/' *.sql
# 去掉脚本中的 date 函数
sed -i 's/ date / /g' *.sql
```

然后需要手动修改一下 1.sql 文件

```
##### d1.sql #####
# 需要 将 where 条件的内容修改
# where
#   l_shipdate <= date '1998-12-01' - interval '90' day (3)

# 修改为 如下
where
    l_shipdate <= '1998-12-01' - interval '90' day
```

4. 进行TPC-H基准测试

首先使用命令进行统一的修改。为每条 SQL 语句指名使用的数据库。

```
# 在个 SQL 文件的第一行 添加 use tpch; 的语句(在queries目录下执行)
sed -i '1a use tpch; \n' *.sql
```

如果执行单独某条query，可以复制执行，或者使用source

```
mysql> source /home/TPC-H/dbgen/queries/d15.sql
```

如果要执行全部的22条query，可以在 queries 文件夹中运行命令：

```
for i in {1..22}; do mysql -u root -p123456 < d$i.sql; done > run.log 2>&1
```

还可以使用提供的脚本，位于dbgen/tpch-benchmark-olap.py

直接python3 tpch-benchmark-olap.py 运行即可，该脚本会在dbgen/logs/下面生成完整的log以及最后提取出来的计时log，部分结果如下：

```
=== SQL Execution Summary ===
Query d1.sql      : 136.891994 seconds
Query d2.sql      : 2.744315 seconds
Query d3.sql      : 248.549623 seconds
Query d4.sql      : 11.283119 seconds
Query d5.sql      : 46.075484 seconds
Query d6.sql      : 27.889816 seconds
Query d7.sql      : 29.506548 seconds
Query d8.sql      : 130.817898 seconds
Query d9.sql      : 86.894261 seconds
Query d10.sql     : 39.125315 seconds
Query d11.sql     : 7.032995 seconds
Query d12.sql     : 40.761182 seconds
Query d13.sql     : 176.769705 seconds
Query d14.sql     : 36.458549 seconds
Query d15.sql     : 66.147650 seconds
Query d16.sql     : 6.522932 seconds
Query d17.sql     : 8.794944 seconds
Query d18.sql     : 38.552912 seconds
Query d19.sql     : 3.651665 seconds
Query d20.sql     : 10.412229 seconds
Query d21.sql     : 97.271048 seconds
Query d22.sql     : 2.869727 seconds

Total Execution Time for Queries: 1255.023911 seconds
Script Total Time (including overhead): 1255.025033 seconds
=== Benchmark Completed ===
```

5. Kingbase/PG导入TPC-H数据

在下载完/使用dbgen生成完数据之后，可以先手动创建数据库的表和外键约束，具体参考(dss.ddl和dss.ri)

然后在tpch_data文件夹下，使用管理员权限在powershell里面运行如下脚本：

这里的脚本主要是因为生成的tbl数据每一行的末尾会有一个“|”，导致PG/kingbase数据库读取时报错，需要将最后一个“|”去掉，这里会创建一个新的tbl文件夹，把处理好的表数据输出到tbl文件夹中，后续直接导入即可

```

# Check and remove the existing tbl folder
if (Test-Path -Path "tbl") {
    Remove-Item -Path "tbl" -Recurse -Force
}

# Create a new tbl folder
New-Item -Path "tbl" -ItemType Directory

# Process all .tbl files
Get-ChildItem *.tbl | ForEach-Object {
    $name = "tbl\$($_.Name)"
    Write-Host "Processing file: $name"

    # Clear the target file
    Set-Content -Path $name -Value ""

    # Remove the trailing '|' character from each line
    Get-Content $_.FullName | ForEach-Object {
        $_ -replace '\|$', ''
    } | Set-Content -Path $name
}

```

在附件scripts文件夹下也提供了这个脚本convert_tbl.ps1，你可以直接运行

```
.\convert_tbl.ps1
```

在kingbase里面创建表，然后按照下面的方法导入即可

```

# 替换你自己的路径
copy part from 'C:\Users\ZhaoyangZhang\Desktop\tpch_data\tbl\part.tbl' with
delimiter as '|' NULL '';
copy region from 'C:\Users\ZhaoyangZhang\Desktop\tpch_data\tbl\region.tbl' with
delimiter as '|' NULL '';
copy nation from 'C:\Users\ZhaoyangZhang\Desktop\tpch_data\tbl\nation.tbl' with
delimiter as '|' NULL '';
copy customer from 'C:\Users\ZhaoyangZhang\Desktop\tpch_data\tbl\customer.tbl'
with delimiter as '|' NULL '';
copy supplier from 'C:\Users\ZhaoyangZhang\Desktop\tpch_data\tbl\supplier.tbl'
with delimiter as '|' NULL '';
copy partsupp from 'C:\Users\ZhaoyangZhang\Desktop\tpch_data\tbl\partsupp.tbl'
with delimiter as '|' NULL '';
copy lineitem from 'C:\Users\ZhaoyangZhang\Desktop\tpch_data\tbl\lineitem.tbl'
with delimiter as '|' NULL '';
copy orders from 'C:\Users\ZhaoyangZhang\Desktop\tpch_data\tbl\orders.tbl' with
delimiter as '|' NULL '';

```

最后验证一下(这里只是导入了 前5个小表来示例)


```

        C_PHONE          CHAR(15) NOT NULL,
        C_ACCTBAL         DECIMAL(15,2) NOT NULL,
        C_MKTSEGMENT      CHAR(10) NOT NULL,
        C_COMMENT         VARCHAR(117) NOT NULL);

CREATE TABLE orders ( O_ORDERKEY          INTEGER NOT NULL,
                       O_CUSTKEY           INTEGER NOT NULL,
                       O_ORDERSTATUS       CHAR(1) NOT NULL,
                       O_TOTALPRICE        DECIMAL(15,2) NOT NULL,
                       O_ORDERDATE         DATE NOT NULL,
                       O_ORDERPRIORITY     CHAR(15) NOT NULL,
                       O_CLERK             CHAR(15) NOT NULL,
                       O_SHIPPRIORITY      INTEGER NOT NULL,
                       O_COMMENT           VARCHAR(79) NOT NULL);

CREATE TABLE lineitem ( L_ORDERKEY        INTEGER NOT NULL,
                         L_PARTKEY          INTEGER NOT NULL,
                         L_SUPPKEY          INTEGER NOT NULL,
                         L_LINENUMBER       INTEGER NOT NULL,
                         L_QUANTITY         DECIMAL(15,2) NOT NULL,
                         L_EXTENDEDPRICE   DECIMAL(15,2) NOT NULL,
                         L_DISCOUNT        DECIMAL(15,2) NOT NULL,
                         L_TAX              DECIMAL(15,2) NOT NULL,
                         L_RETURNFLAG       CHAR(1) NOT NULL,
                         L_LINESTATUS       CHAR(1) NOT NULL,
                         L_SHIPDATE         DATE NOT NULL,
                         L_COMMITDATE      DATE NOT NULL,
                         L_RECEIPTDATE     DATE NOT NULL,
                         L_SHIPINSTRUCT    CHAR(25) NOT NULL,
                         L_SHIPMODE         CHAR(10) NOT NULL,
                         L_COMMENT          VARCHAR(44) NOT NULL);

```

如果是Opengauss则同MySQL，因为Opengauss完全兼容MySQL语法。